

Practical Work 6: GlusterFS

Le Quy Ton
23BI14425

December 15, 2025

1 Introduction

GlusterFS is a scalable network-attached storage file system. This practical work aims to install GlusterFS, configure a distributed replicated volume, and evaluate its performance through benchmarking using Python automation.

2 Setup and Configuration

2.1 Installation

We installed GlusterFS server on our Debian environment using the following commands:

```
1 sudo apt-get update
2 sudo apt-get install -y glusterfs-server
3 sudo systemctl start glusterd
4 sudo systemctl enable glusterd
```

2.2 Trusted Pool and Volume Creation

We used a Python automation script to detect the machine's real IP address (as GlusterFS prohibits 'localhost' for volume creation) and configured a **Distributed Replicated Volume** (Replica count = 2).

1. Prepare Bricks:

```
1 mkdir -p /data/brick1/gv0
2 mkdir -p /data/brick2/gv0
```

2. Create Volume:

```

1 # Using the machine's Real IP (detected automatically)
2 sudo gluster volume create gv0 replica 2 \
3     <Host_IP>:/data/brick1/gv0 \
4     <Host_IP>:/data/brick2/gv0 force

```

3. Start and Mount Volume:

```

1 sudo gluster volume start gv0
2 mkdir -p /mnt/gluster_client
3 # Mounting using the Real IP
4 sudo mount -t glusterfs <Host_IP>:/gv0 /mnt/gluster_client

```

3 Benchmarking

We utilized a Python script to automate the benchmarking process. The tests focused on two scenarios required by the specifications.

3.1 Methodology

- **Large Files:** We generated a 512MB file filled with zeros. We cleared the RAM cache (`drop_caches`) before reading to ensure we measured disk I/O, not RAM speed. The metric is Read Speed (MB/s).
- **Small Files:** We created 500 small empty files sequentially in the GlusterFS mount point. The metric is Number of Accesses (Creations) per second.

3.2 Results

The benchmarks were performed on a single-node setup (simulating 2 bricks). The results obtained from the automation script are shown below.

Configuration	Small Files (ops/s)	Large Files Read (MB/s)
1 Server (2 Bricks)	190.28	215.17
<i>Hypothetical 2 Servers</i>	<i>Estimated 160</i>	<i>Estimated 400</i>

Table 1: Performance Benchmarks

```
=== BENCHMARK 2: SMALL FILES (Accesses/s) ===
-> Creating 500 small files...
-> Result: 190.28 ops/s

=====
RESULTS FOR REPORT
=====
Large Files Read: 215.17 MB/s
Small Files Rate: 190.28 ops/s
```

Figure 1: Successful execution and benchmark results

4 Conclusion

We successfully set up a GlusterFS environment with a distributed replicated volume using an automated Python script to handle IP detection and configuration.

- **Replication** ensures data availability; if one brick fails, data is still accessible.
- **Performance:** We achieved a read speed of **215.17 MB/s** for large files and a creation rate of **190.28 ops/s** for small files. The overhead for small files is expected due to metadata synchronization in a distributed file system.